

Team Productivity Application

Post-Review Final — Incorporates all Architecture Review Checklist fixes

Version: 1.1 (Post-Review) **Date:** July 2026 **Status:** Approved for Build

React 18

Node.js 20 LTS

PostgreSQL 16

Express 4

JWT Auth

REST API

TABLE OF CONTENTS

1

Tech Stack

4

API Design

2

System Components

5

Implementation Sequence

3

Data Model

6

Top 3 Risks

1

Tech Stack

LAYER	TECHNOLOGY	VERSION	PURPOSE
Frontend Framework	React	18.3.x	Component-based UI
Build Tool	Vite	5.x	Dev server and bundler
Frontend Routing	React Router DOM	6.x	Client-side routing and protected routes
HTTP Client	Axios	1.x	API calls with JWT interceptor and 401 redirect
Drag and Drop	@dnd-kit/core	6.x	Kanban card drag interactions
Backend Runtime	Node.js	20 LTS	JavaScript server runtime
Backend Framework	Express	4.x	REST API routing and middleware
Database	PostgreSQL	16.x	Relational data store
DB Query Layer	node-postgres (pg)	8.x	Raw SQL queries — no ORM
Password Hashing	bcryptjs	2.x	Credential storage (min 10 salt rounds)
Auth Tokens	jsonwebtoken	9.x	JWT sign and verify (HS256, 24h expiry)
Rate Limiting	express-rate-limit	7.x	Login brute-force protection (10/5min per IP)
CORS	cors	2.x	Cross-origin policy (allow client origin only)
Env Config	dotenv	16.x	Environment variable loading

LAYER	TECHNOLOGY	VERSION	PURPOSE
Dev Reload	nodemon	3.x	Backend auto-restart in development

2 System Components

BACKEND

COMPONENT	FILE PATH	RESPONSIBILITY	COMMUNICATES WITH
Entry Point	<code>server/index.js</code>	Boots Express, loads middleware, mounts all routers	All routers, middleware
DB Connection	<code>server/config/db.js</code>	Exports a pg Pool; all queries run through this single pool	PostgreSQL
Auth Middleware	<code>server/middleware/auth.js</code>	Verifies JWT on every protected request; attaches <code>req.user</code>	jsonwebtoken, all protected routes
PM Guard	<code>server/middleware/requirePM.js</code>	Returns 403 if <code>req.user.role !== 'pm'</code>	Auth middleware, PM-only routes
Rate Limiter	<code>server/middleware/rateLimiter.js</code>	Limits login to 10 attempts per IP per 5 minutes	express-rate-limit, auth routes
Auth Router	<code>server/routes/auth.routes.js</code>	Mounts POST <code>/register</code> and POST <code>/login</code>	Auth controller
Team Router	<code>server/routes/team.routes.js</code>	Mounts GET <code>/team</code> and GET <code>/team/invite-code</code>	Team controller
Sprint Router	<code>server/routes/sprint.routes.js</code>	Mounts sprint CRUD and complete endpoints	Sprint controller
Task Router	<code>server/routes/task.routes.js</code>	Mounts backlog, sprint tasks, and task CRUD endpoints	Task controller
Retro Router	<code>server/routes/retro.routes.js</code>	Mounts retro submit, status, results, and close	Retro controller
Auth Controller	<code>server/controllers/auth.controller.js</code>	Two handlers: <code>registerPM</code> (creates team) and <code>registerDeveloper</code> (joins	db.js, bcryptjs, jsonwebtoken

COMPONENT	FILE PATH	RESPONSIBILITY	COMMUNICATES WITH
		team); plus <code>login</code> FIXED	
Team Controller	<code>server/controllers/team.controller.js</code>	Returns team info, member list, and invite code	db.js
Sprint Controller	<code>server/controllers/sprint.controller.js</code>	Handles sprint create, delete, complete (atomic transaction with <code>team_member_count</code> snapshot) FIXED	db.js
Task Controller	<code>server/controllers/task.controller.js</code>	Handles task CRUD including sprintId assignment and status updates	db.js
Retro Controller	<code>server/controllers/retro.controller.js</code>	Handles submission, status check, results (no <code>user_id</code> in output), and close	db.js

FRONTEND

COMPONENT	FILE PATH	RESPONSIBILITY	COMMUNICATES WITH
App Root	<code>client/src/App.jsx</code>	Defines all routes; wraps app in <code>AuthProvider</code>	<code>AuthContext</code> , <code>React Router</code> , all pages
Auth Context	<code>client/src/context/AuthContext.jsx</code>	Holds decoded user and JWT; exposes login/logout helpers	<code>localStorage</code> , all pages
API Service	<code>client/src/services/api.js</code>	Axios instance with JWT header injected on every request; redirects to login on 401	<code>AuthContext</code> , all pages/components
Protected Route	<code>client/src/components/ProtectedRoute.jsx</code>	Redirects unauthenticated users to <code>/login</code>	<code>AuthContext</code> , <code>React Router</code>
PM Route Guard	<code>client/src/components/PMRoute.jsx</code>	Redirects Developers away from PM-only pages (403 prevention)	<code>AuthContext</code> , <code>React Router</code>

COMPONENT	FILE PATH	RESPONSIBILITY	COMMUNICATES WITH
Login Page	<code>client/src/pages/LoginPage.jsx</code>	Email/password form; sets JWT in context on success	api.js, AuthContext
Register Page	<code>client/src/pages/RegisterPage.jsx</code>	Role selector toggles invite code field; calls correct register path	api.js, AuthContext
Dashboard Page	<code>client/src/pages/DashboardPage.jsx</code>	Single data-owner for sprint + task state; delegates rendering to one of three child views based on sprint status FIXED	api.js, ActiveSprintView, SprintCompleteView, NoSprintView
Active Sprint View	<code>client/src/views/ActiveSprintView.jsx</code>	Renders KanbanBoard; receives tasks as props; calls status-update callback on card drop FIXED	KanbanBoard, TaskCard
Sprint Complete View	<code>client/src/views/SprintCompleteView.jsx</code>	Renders retro prompt banner and Done task list; links to retro form FIXED	api.js
No Sprint View	<code>client/src/views/NoSprintView.jsx</code>	Renders empty-state prompt with Create Sprint button (PM only) FIXED	AuthContext
Kanban Board	<code>client/src/components/KanbanBoard.jsx</code>	Controlled component — receives tasks and onStatusChange as props; owns drag-and-drop interaction only FIXED	@dnd-kit, KanbanColumn, TaskCard
Kanban Column	<code>client/src/components/KanbanColumn.jsx</code>	Single droppable column; renders task cards; shows count badge and empty state	@dnd-kit, TaskCard
Task Card	<code>client/src/components/TaskCard.jsx</code>	Draggable card showing title, assignee initials,	@dnd-kit, TaskModal

COMPONENT	FILE PATH	RESPONSIBILITY	COMMUNICATES WITH
		priority badge; opens TaskModal on click	
Task Modal	<code>client/src/components/TaskModal.jsx</code>	Read-only task detail; PM sees Edit and Delete buttons; posts changes via callback to parent	api.js
Add To Sprint Modal	<code>client/src/components/AddToSprintModal.jsx</code>	Multi-select task list with confirm; calls PATCH task endpoint with sprintId FIXED	api.js
Backlog Page	<code>client/src/pages/BacklogPage.jsx</code>	Lists backlog tasks; opens TaskModal for create/edit; opens AddToSprintModal for sprint assignment	api.js, TaskModal, AddToSprintModal
Sprint Create Page	<code>client/src/pages/SprintCreatePage.jsx</code>	Form to create sprint — name and end date (PM only)	api.js
Retro Form Page	<code>client/src/pages/RetroFormPage.jsx</code>	Three-question anonymous form; shows Done tasks above; submitted state on reload	api.js
Retro Results Page	<code>client/src/pages/RetroResultsPage.jsx</code>	PM-only two-panel layout — tasks left, responses right; Close Retro button	api.js
Sprint History Page	<code>client/src/pages/SprintHistoryPage.jsx</code>	PM-only list of completed sprints with stats; click to open retro results	api.js
Team Page	<code>client/src/pages/TeamPage.jsx</code>	PM-only; member table and copyable invite code	api.js

3 Data Model

ENTITY RELATIONSHIP

```

teams    —< users      one team has many users
teams    —< sprints     one team has many sprints
teams    —< tasks       one team has many tasks
sprints  —< tasks       one sprint has many tasks (sprint_id nullable)
sprints  —< retro_submissions one sprint has many submissions
users    —< tasks       one user assigned to many tasks (assignee_id nullable)
users    —< retro_submissions one user has many submissions (tracking only)

```

TEAMS

FIELD	TYPE	CONSTRAINTS
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
name	VARCHAR(100)	NOT NULL
invite_code	CHAR(6)	UNIQUE NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()

Indexes: UNIQUE(invite_code)

USERS

FIELD	TYPE	CONSTRAINTS
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
name	VARCHAR(100)	NOT NULL
email	VARCHAR(255)	UNIQUE NOT NULL
password_hash	TEXT	NOT NULL
role	VARCHAR(20)	NOT NULL, CHECK IN ('pm','developer')
team_id	UUID	FK → teams(id), nullable until team joined
created_at	TIMESTAMP	DEFAULT NOW()

Indexes: UNIQUE(email), INDEX(team_id)

SPRINTS

FIELD	TYPE	CONSTRAINTS
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
team_id	UUID	FK → teams(id) NOT NULL
name	VARCHAR(100)	NOT NULL
goal	TEXT	nullable

FIELD	TYPE	CONSTRAINTS
start_date	DATE	NOT NULL
end_date	DATE	NOT NULL
status	VARCHAR(20)	DEFAULT 'active', CHECK IN ('active','completed')
retro_closed	BOOLEAN	DEFAULT FALSE
team_member_count	INTEGER	NOT NULL DEFAULT 0 — snapshot at sprint close time FIXED
created_at	TIMESTAMP	DEFAULT NOW()

Indexes: INDEX(team_id), INDEX(team_id, status) — for active sprint lookup

TASKS

FIELD	TYPE	CONSTRAINTS
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
team_id	UUID	FK → teams(id) NOT NULL
sprint_id	UUID	FK → sprints(id), nullable (null = backlog)
title	VARCHAR(100)	NOT NULL
description	TEXT	nullable
assignee_id	UUID	FK → users(id), nullable
priority	VARCHAR(10)	DEFAULT 'medium', CHECK IN ('low','medium','high')
status	VARCHAR(20)	DEFAULT 'backlog', CHECK IN ('backlog','todo','in_progress','done')
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Indexes: INDEX(team_id), INDEX(sprint_id), INDEX(sprint_id, status)

RETRO_SUBMISSIONS

FIELD	TYPE	CONSTRAINTS
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
sprint_id	UUID	FK → sprints(id) NOT NULL
user_id	UUID	FK → users(id) NOT NULL — tracking only, never returned to client
answer_1	TEXT	NOT NULL
answer_2	TEXT	NOT NULL

FIELD	TYPE	CONSTRAINTS
answer_3	TEXT	NOT NULL
submitted_at	TIMESTAMP	DEFAULT NOW()
UNIQUE(sprint_id, user_id)		

Indexes: INDEX(sprint_id), UNIQUE(sprint_id, user_id)

Tenancy Rule (all controllers): Every SELECT, UPDATE, and DELETE query on sprints, tasks, and retro_submissions must include `AND team_id = $req.user.teamId` in the WHERE clause. This prevents cross-team data access for any authenticated user regardless of role. **FIXED**

4 API Design

All error responses: { "error": "message", "field": "fieldName" } — field only on validation errors. Auth badges:

JWT = requires valid token **PM Only** = JWT + role=pm

AUTHENTICATION

POST /api/auth/register

No Auth

REQUEST BODY

```
{ "name": string,
  "email": string,
  "password": string,      // min 8 chars
  "role": "pm"|"developer",
  "inviteCode": string     // required if developer
}
```

RESPONSE 201

```
{ "token": "<jwt>",
  "user": {
    "id", "name", "email",
    "role", "teamId"
  }
}
```

Errors: 400 validation | 409 email exists | 400 invalid invite code | 409 team already has PM

POST /api/auth/login

No Auth · Rate Limited

REQUEST BODY

```
{ "email": string,
  "password": string
}
```

RESPONSE 200

```
{ "token": "<jwt>",
  "user": {
    "id", "name", "email",
    "role", "teamId"
  }
}
```

Errors: 401 invalid credentials | 429 rate limit. Logout: client-side only — clear JWT from localStorage, reset AuthContext, redirect to /login. No server endpoint needed.

TEAM

GET /api/team

JWT

PARAMS

None – team_id from req.user

RESPONSE 200

```
{ "team": { "id","name","createdAt" },
  "members": [{
    "id","name","role","createdAt"
  }]
}
```

GET /api/team/invite-code

PM Only

PARAMS

None

RESPONSE 200

```
{ "inviteCode": "A1B2C3" }
```

SPRINTS

GET /api/sprints/active

JWT

PARAMS

None

RESPONSE 200

```
{ "sprint": {
  "id","name","goal",
  "startDate","endDate",
  "status","retroClosed"
} | null
}
```

GET /api/sprints/history

PM Only

PARAMS

None

RESPONSE 200

```
{ "sprints": [{
  "id","name","startDate","endDate",
  "tasksTotal","tasksDone",
  "retroTotal","retroSubmitted"
}]
}
```

POST /api/sprints

PM Only

REQUEST BODY

```
{ "name": string,
  "goal": string, // optional
  "endDate": "YYYY-MM-DD"
}
```

RESPONSE 201

```
{ "sprint": {
  "id","name","goal",
  "startDate","endDate","status"
}
}
```

Errors: 400 validation | 409 active sprint exists | 409 retro still open

DELETE /api/sprints/:id

PM Only

PARAMS

:id – sprint UUID

RESPONSE 204

No content

Errors: 403 | 404 | 409 sprint has tasks | Tenancy: WHERE id=\$id AND team_id=\$teamId

PATCH /api/sprints/:id/complete

PM Only

REQUEST BODY

{}

 // no body required

RESPONSE 200

```
{ "sprint": {
  "id", "status": "completed"
},
  "tasksReturned": number
}
```

Atomic transaction: (1) sprint.status→completed, (2) sprint.team_member_count = COUNT(users WHERE team_id), (3) tasks WHERE sprint_id AND status IN (todo,in_progress) → backlog + sprint_id=null. Errors: 404 | 409 already completed

TASKS

GET /api/tasks/backlog

JWT

PARAMS

None – limit 100, order by created_at DESC

RESPONSE 200

```
{ "tasks": [{
  "id", "title", "description",
  "priority", "status",
  "assignee": {"id", "name"} | null,
  "createdAt"
}]
}
```

GET /api/tasks/sprint/:sprintId

JWT

PARAMS

:sprintId – sprint UUID

RESPONSE 200

```
{ "tasks": [{
  "id", "title", "description",
  "priority", "status",
  "assignee": {"id", "name"} | null
}]
}
```

Tenancy: JOIN sprints WHERE sprints.team_id = req.user.teamId

POST /api/tasks

PM Only

REQUEST BODY

```
{ "title": string,
  "description": string, // optional
  "assigneeId": UUID,    // optional
  "priority": "low"|"medium"|"high",
  "sprintId": UUID       // optional
}
```

RESPONSE 201

```
{ "task": { ...all fields } }
```

If sprintId provided, status defaults to 'todo'; otherwise 'backlog'

PATCH /api/tasks/:id

PM Only

REQUEST BODY (ANY SUBSET)

```
{ "title": string,
  "description": string,
  "assigneeId": UUID | null,
  "priority": string,
  "sprintId": UUID | null // ← add to sprint / remove
}
```

RESPONSE 200

```
{ "task": { ...updated fields } }
```

sprintId field added — fixes checklist gap. Setting sprintId→UUID sets status→todo. Setting sprintId→null sets status→backlog.

PATCH /api/tasks/:id/status

JWT (PM + Dev)

REQUEST BODY

```
{ "status": "todo"|"in_progress"|"done" }
```

RESPONSE 200

```
{ "task": {
  "id", "status", "updatedAt"
}
```

Errors: 400 invalid status | 403 sprint is completed (board locked) | 404

DELETE /api/tasks/:id

PM Only

PARAMS

:id — task UUID

RESPONSE 204

No content

Errors: 403 | 404 | 409 task is in an active sprint

RETROSPECTIVE

POST /api/retro/:sprintId

JWT

REQUEST BODY

RESPONSE 201

```
{ "answer1": string, // 10-500 chars
  "answer2": string,
  "answer3": string
}
```

```
{ "message": "Submission recorded." }
```

user_id stored in DB; never returned. Errors: 400 validation | 403 sprint not completed | 409 already submitted | 404

GET `/api/retro/:sprintId/status`

JWT

PARAMS

`:sprintId` — sprint UUID

RESPONSE 200

```
{ "submitted": boolean }
```

GET `/api/retro/:sprintId/results`

PM Only

RESPONSE 200

```
{ "sprint": { "id", "name", "retroClosed" },
  "participation": {
    "submitted": number,
    "total": number // from team_member_count
  },
  "tasks": [{ "id", "title", "priority",
    "assignee": { "name" } | null }],
  "responses": {
    "question1": [ "text", "text", ... ],
    "question2": [ ... ],
    "question3": [ ... ]
  }
}
```

user_id excluded at query level — SELECT uses COUNT(DISTINCT user_id) for participation only. total reads sprint.team_member_count (snapshot). Errors: 403 | 404

PATCH `/api/retro/:sprintId/close`

PM Only

REQUEST BODY

```
{}
```

 // no body required

RESPONSE 200

```
{ "message": "Retrospective closed." }
```

Sets sprints.retro_closed = true. Unblocks new sprint creation. Errors: 403 | 404 | 409 already closed

5 Implementation Sequence

Phases 2 and 3 can run in parallel. Each phase builds only on fully completed prior phases.

1 Foundation

Database + project skeletons

- ▶ Initialise `server/` (Node.js) and `client/` (Vite + React) project directories
- ▶ Configure `server/.env` : `DATABASE_URL`, `JWT_SECRET`, `PORT`, `CLIENT_URL`
- ▶ Create `server/config/db.js` — pg Pool with connection string
- ▶ Write and run all `teams` → `users` → `sprints` (with `tasks` → `retro_submissions`
`CREATE TABLE` `team_member_count`)
migrations in order: →
- ▶ Verify all FK constraints, CHECK constraints, UNIQUE indexes, and tenancy fields with a seed script
- ▶ Set up Express entry point, CORS (allow client origin only), and JSON body parser

2 Core API

All endpoints — test with Postman before touching frontend

- ▶ Auth — `registerPM` , `registerDeveloper` , `login` ; JWT generation; rate limiter on login
- ▶ Team — GET members, GET invite code
- ▶ Tasks — full CRUD including `sprintId` assignment; status update endpoint
- ▶ Retro — submit, status check, results (verify `user_id` absent in response); close
- ▶ Sprint — create, delete, GET active, GET history
- ▶ Sprint complete **last** : atomic transaction (status + `team_member_count` snapshot + task reset); test all three transaction — **step** together only after tasks and retro endpoints are verified independently
- ▶ Apply auth middleware and PM guard to all routes; verify tenancy WHERE clause on every resource query
- ▶ Add input validation to all POST/PATCH handlers with field-level error responses

3 Frontend Skeleton

All pages render with mock data — can run parallel to Phase 2

- ▶ Set up React Router with all routes in `App.jsx`
- ▶ Build `AuthContext` with login/logout and `localStorage` persistence
- ▶ Build `ProtectedRoute` and `PMRoute` guard components
- ▶ Build all page shells — Login, Register, Dashboard, Backlog, Sprint Create, Retro Form, Retro Results, Sprint History, Team
- ▶ Build `ActiveSprintView` , `SprintCompleteView` , `NoSprintView` — Dashboard delegates to one of three based on mock sprint state
- ▶ Build `KanbanBoard` as controlled component with @dnd-kit using hardcoded task arrays — verify drag works; `KanbanBoard` calls `onStatusChange` prop, does not call `api.js` directly
- ▶ Build `TaskCard` , `TaskModal` , `AddToSprintModal` components

4 Integration

Connect frontend to live API — requires Phases 2 and 3 complete

- ▶ Configure `services/api.js` : base URL, JWT header interceptor, 401 → logout + redirect (implement this Axios in `first` — every subsequent step depends on it)
- ▶ Wire Login and Register pages to auth endpoints
- ▶ Wire Dashboard: fetch GET `/api/sprints/active` on mount; pass result to correct child view

- ▶ Wire ActiveSprintView: fetch GET /api/tasks/sprint/:id; pass tasks to KanbanBoard; handle onStatusChange → PATCH /api/tasks/:id/status
- ▶ Wire "Complete Sprint" button with confirmation dialog → PATCH /api/sprints/:id/complete; refresh Dashboard state on success
- ▶ Wire Backlog page: load tasks, create/edit/delete via TaskModal; add-to-sprint via AddToSprintModal → PATCH /api/tasks/:id with sprintId
- ▶ Wire Retro Form page: check GET /api/retro/:sprintId/status on load; POST on submit
- ▶ Wire Retro Results page: GET results; Close Retro → PATCH close
- ▶ Wire Sprint History and Team pages
- ▶ End-to-end test of full demo flow: register PM → invite dev → create sprint → add tasks → move cards → close sprint → submit retro → view results

5 Polish

Error handling, empty states, validation — all user-visible gaps

- ▶ Add client-side inline validation to all forms with per-field error messages matching server responses
- ▶ Add loading spinners to all API-dependent renders
- ▶ Add empty states to all list views: backlog, board columns, sprint history
- ▶ Add toast notification system for server errors (card move failure, 409 conflicts, etc.)
- ▶ Verify server-side validation returns field-level errors on all POST/PATCH handlers
- ▶ Verify rate limiter returns 429 and frontend shows appropriate message
- ▶ Inspect network tab: confirm user_id is absent from all retro API responses
- ▶ Cross-browser test on Chrome, Firefox, and Edge

6 Top 3 Architectural Risks

Risk 1 — High Sprint Close Transaction Failure Leaves Ambiguous UI State

The sprint close operation updates three things atomically: sprint status, team_member_count snapshot, and task resets. PostgreSQL will roll back cleanly on DB failure. However, if Express crashes after the transaction commits but before it returns 200 to the client, the frontend will not know the sprint closed. The user clicks "Complete Sprint" again and receives a 409 (already completed) — which looks like an error rather than a success confirmation.

Mitigation: The frontend must treat HTTP 409 on sprint close as a success condition. On any 409, fetch GET /api/sprints/active immediately — if it returns null (no active sprint), the close succeeded; re-render accordingly and show an informational message, not an error toast.

Risk 2 — High JWT Expiry Causes Silent Data Loss on Kanban Board

JWT expires after 24 hours with no refresh token. If a user leaves the app open overnight and resumes the next morning, every API call returns 401. Without a correctly wired Axios interceptor, PATCH /api/tasks/:id/status calls will fail silently — cards move visually but the status is never persisted. The developer believes the board updated; it did not.

Mitigation: Implement the Axios 401 interceptor in Phase 4 Step 1 — before any other integration work. It must: clear JWT from localStorage, call AuthContext logout, and redirect to /login with "Session expired" message. This is the single highest-priority item in Phase 4.

Risk 3 — Medium **Concurrent Card Moves With No Conflict Detection**

The architecture accepts last-write-wins for concurrent Kanban updates (no WebSockets, no conflict detection). Two developers moving the same card simultaneously will silently overwrite each other. Neither sees an error. The PM watching the board during a standup sees a snapshot that may not reflect the current state. This is the most user-visible risk during a multi-user demo.

Mitigation: Add a "Last refreshed X seconds ago" label and a manual refresh button to the Kanban board header. This sets an honest expectation that the board is a snapshot. Additionally, display a subtle "updated_at" timestamp on each card in the detail modal so users can detect stale moves. Do not promise live behaviour the architecture cannot deliver.